AAI 520 Team 2 Investment Research Agent: Multi-Agent Financial Analysis System *Supplemental Report to the Code Notebook*

Item	Details
Course	AAI 520 Natural Language Processing and GenAI, M.S. Applied Artificial Intelligence, University of San Diego
Assignment	Final Team Project: Multi-Agent Financial Analysis System (October 19, 2026)
Team	Team 2: Deepika Shrestha, Christian Lopez, Cameron Aljilani
Submission	Code Notebook (HTML) plus this Supplemental Report
Repository	https://github.com/dshrestha37/investment-research-agent
Commit / notebook	Latest commit 245c187 (README update, October 18, 2026, 11:14 AM PT); executed notebook last pushed in 9958c03 (5:30 PM PT). Notebook: AAI520_Final_Project_Team_2_Aljilani_Lopez_Shrestha.ipynb
Ticker / LLM	AAPL (set once in the notebook TICKER cell) Google Gemini gemini-3.8-flash (via GEMINI_MODEL)
Run date	Sunday, October 18, 2026, ~9:11 AM PT (Run 1 audit id 20261018T161147-1109c4)

1. Overview

This is the supplemental report for the Investment Research Agent built for the AAI 520 Final Project. We built this tool to be universal: provide one valid stock symbol and it plans the research, pulls data with tools, runs a news prompt chain, routes the results to three specialists, drafts a short analyst note, scores and sometimes revises that note, and saves what it learned for the next run. Every number is tagged with a source id using [S#] for a tool result or [N#] for a news article. This notebook runs in Google Colab using only API calls and model inference. A model was not trained as part of this project.

Two rules are enforced everywhere in the code: (1) if a tool returns nothing, the system reports the gap and does not invent an article, metric, or citation; (2) the note also never issues a BUY or SELL rating. Google Gemini LLM use is optional. Every LLM step has a deterministic rule fallback, so the notebook still finishes a complete note even without a Gemini API key. The graded run used the LLM path for every step.

2. Architecture and Overall Structure

The notebook is one Colab wrapper integrating tools and agents created by two teammates.

Notebook section	What it does
Section 0	Initialization; sets the stock ticker.
Section 1	Installs the required libraries.
Section 2	Copies Colab secrets into the environment and indicates which ones are loaded using Boolean (True/False) for each requested key.
Section 3	Writes twelve modules with %%writefile. Each module starts with a change-log docstring so the teammates and evaluators can follow the evolution of the module.
Section 4	Runs ten offline tests that need no API keys or network, so the rule-based path is checked without Gemini.
Sections 5 and 6	Runs the agent twice against the same memory file. This demonstrates how lessons are learned and implemented across runs.
Section 6b	Prints the evidence from both runs: plan, chain stage log, routing log, evaluator history, reflection, and lessons.
Section 6c	Shows three charts: closing price with 20- and 50-day averages, evaluator scores for each draft in Runs 1 and 2, and how many Run 1 articles were routed to each specialist.

Notebook section	What it does
Section 7	Builds a FAISS side index. This index is not part of the workflow.
Section 8	Summarizes the team's results.

Three contracts hold the modules together:
ToolResult (schema.py). Every tool returns the same record (tool, ok, data, error, warnings, source, fetched_at) and never raises, so a failed tool becomes a data gap instead of a crash.
Source IDs. Each executed tool result is numbered [S1], [S2], … in the order it ran. recall_memory always runs first, so memory is cited as [S1]. The chain numbers the articles it keeps [N1], [N2], … The chain's summary gate and the evaluator reject any other ID.
State. MemoryStore keeps the last 20 runs and the current lessons per ticker in ira_state/memory.json. Each run also writes an audit JSON to ira_state/runs/.

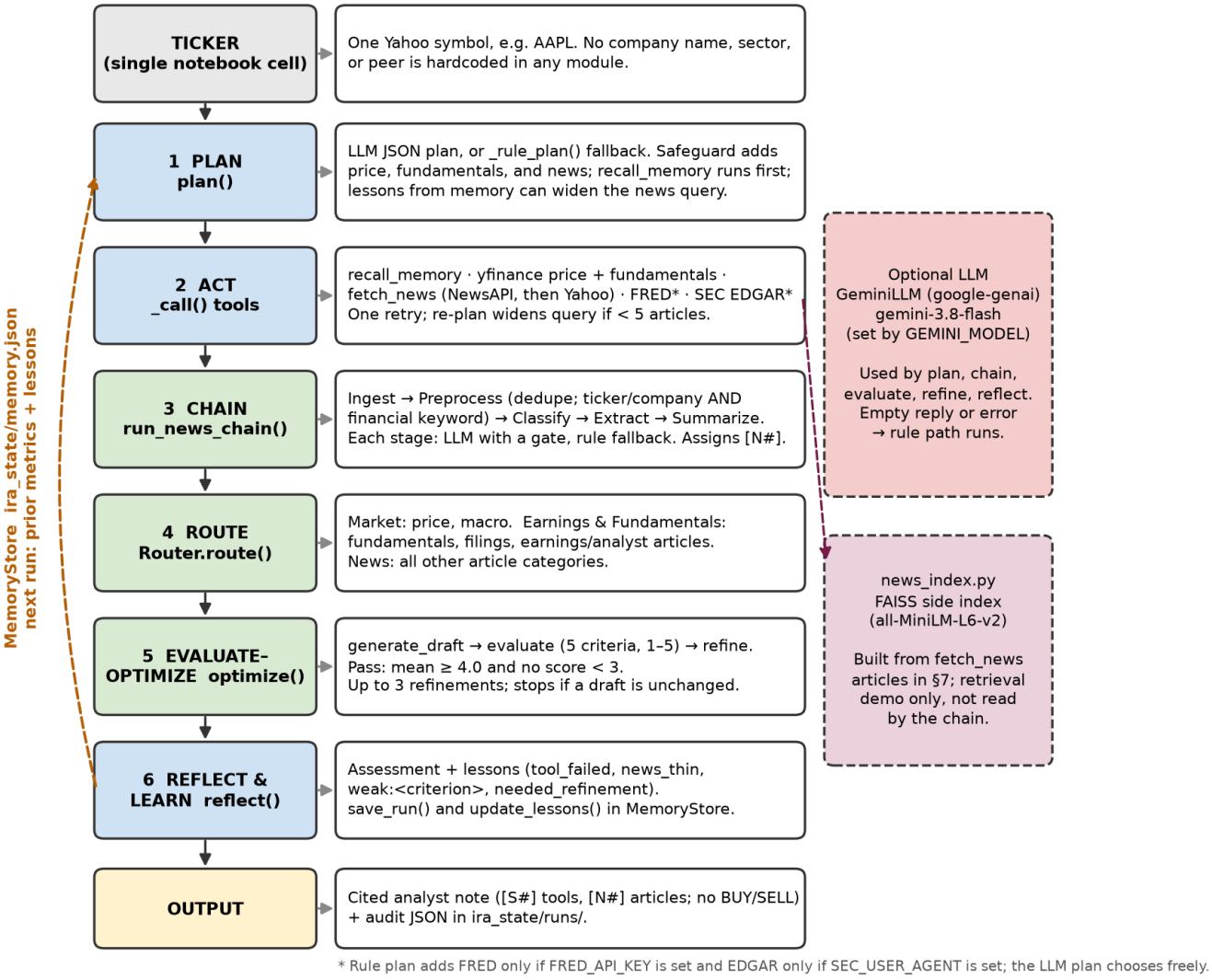


Figure 1. Architecture of ResearchAgent.run() in research_agent.py. Solid arrows are the per-run pipeline; the dashed line is the memory that carries lessons and metrics into the next run.

Table 1 lists the modules and who wrote and changed each one.

Module	Purpose	Origin and integration changes
schema.py	ToolResult contract shared by every tool	Deepika; reviewed, no behavior change
market_tools.py	yfinance price history (returns, 20/50-day averages, range, volatility) and fundamentals (market cap, P/E, EPS, next earnings)	Deepika; reviewed, no behavior change
news_tools.py	NewsAPI fetch, Yahoo headline fetch, category tag, financial-relevance filter, fetch_news wrapper	Deepika (NewsAPI fetch, keyword categories). Christian (Yahoo Finance fetch and parsing, including handling of old and new Yahoo response formats; now fetch_yahoo_news(), the agent's backup source when NewsAPI fails; his earnings vs. general split carried into the category tags). Cameron merged the two, added the relevance filter, and removed the synthetic fallback article.
extra_tools.py	FRED macro series (FEDFUNDS, DGS10, CPIAUCSL) and SEC EDGAR filings (10-K, 10-Q, 8-K)	Deepika; reviewed, no behavior change
llm.py	GeminiLLM and ClaudeLLM wrappers, make_llm, tolerant parse_json	Deepika; Cameron corrected the Claude id, made the Gemini model configurable, raised the token limit, added truncation detection
memory.py	MemoryStore: runs, lessons, metric diff between runs	Deepika; Cameron added lesson de-duplication
chain.py	Workflow 1, prompt chain	Deepika; Cameron added the two-signal relevance gate and combined-citation parsing
router.py	Workflow 2, router and three specialists	Deepika; Cameron fixed the position phrase and the empty-news headline
evaluator.py	Workflow 3, draft, evaluate, refine	Deepika; Cameron fixed heading, citation, and bullet parsing, truncation handling, and the writer's prior-run context
research_agent.py	Orchestrator: plan, act, re-plan, chain, route, optimize, reflect, learn	Deepika; Cameron switched news to fetch_news (NewsAPI, then Yahoo)
news_index.py	FAISS side index over fetched articles; retrieval returns L2 distance	Christian (design and core logic from News_Ingestion_Pipeline.ipynb: 300-character chunks with 50-character overlap, source tags on every chunk, MiniLM embeddings, FAISS index, source-cited retrieval, save and reload). Cameron converted it to a module, connected it to fetch_news output, and removed the zero-shot classifier and the fixed fallback article.
test_pipeline.py	Ten offline tests (no keys, no network)	Deepika; Cameron added three regression tests and updated one assertion

Table 1. Modules written in Section 3 of the notebook.

3. Rubric Map

Table 2 links each required agent function and workflow pattern to the code that implements it and the notebook output that demonstrates it. The output evidence is from the graded Gemini run.

Requirement	Module / function	Notebook location	Output evidence (graded run)
Agent Function 1: Plans research steps	research_agent.plan(), _rule_plan(), MANDATORY safeguard, _followups() re-plan	Section 3 research_agent.py; Section 6b	LLM plan with a reason per step: 6 steps in each run (Run 2 called EDGAR before news); widened-news re-plan not needed (6 relevant articles)
Agent Function 2: Uses tools dynamically	default_tools(), _call(); market_tools, news_tools.fetch_news, extra_tools, recall_memory; news_index	Section 3 modules; Section 5 note; Section 7	Run 1: [S1] memory, [S2] fundamentals, [S3] prices (1y), [S4] NewsAPI (12 articles), [S5] EDGAR, [S6] FRED; FAISS: 20 chunks from 12 articles
Agent Function 3: Self-reflects	evaluator.evaluate() / rule_evaluate(); research_agent.reflect()	Section 3 evaluator.py, research_agent.py; Section 6b	Scores and feedback per draft; Run 1 reflection: "data limitations and baseline memory conditions were clearly delineated"
Agent Function 4: Learns across runs	MemoryStore.save_run(), update_lessons(), diff(); lessons in plan() and compact(); news_thin in _rule_plan()	Section 3 memory.py; Section 6; Section 6b	Run 1 saved 3 LLM lessons; Run 2's plan and note used them; Run 2 Changes since last run: "No change in tracked metrics since run 20261018T161147-1109c4 [S1]"
Workflow 1: Prompt chaining	chain.run_news_chain(): ingest, preprocess, classify, extract, summarize	Section 3 chain.py; Section 6b	12 ingested → 12 after dedupe → 6 kept; classify, extract, summarize all by LLM (0 rule fallbacks)
Workflow 2: Routing	router.Router.route(), TOOL_ROUTES, ARTICLE_ROUTES, three specialists	Section 3 router.py; Section 6b; Figure 2 right	Run 1: Market S3, S6; Earnings S2, S5, N3, N5; News N1, N2, N4, N6. Run 2: same split (EDGAR is S4)
Workflow 3: Evaluator–optimizer	evaluator.optimize(), generate_draft(), evaluate(), refine()	Section 3 evaluator.py; Section 6b; Figure 2 middle	Both runs: 1 evaluation, first draft 5.0 on every criterion, passed; refine not needed. Earlier full run (Oct 17): Run 2 refined 4.4 → 5.0
Offline verification	test_pipeline.py (10 tests)	Section 4	Ran 10 tests in 0.117s: OK

Table 2. Rubric requirements mapped to code and output evidence.

4. Agent Functions and Capabilities

4.1 Planning

plan() sends the LLM the tool catalog, the stored lessons, and whether a prior run exists, then asks for a JSON list of steps (tool, arguments, reason). Unknown tools are dropped. If no valid plan comes back, _rule_plan() builds one: recall_memory, get_price_history, get_fundamentals, fetch_news, then get_macro if a FRED key is present and get_edgar_filings if SEC_USER_AGENT is set. Those two conditions only apply to the rule plan; the LLM planner can choose any tool. A safeguard appends any missing mandatory tool (price, fundamentals, news) and puts recall_memory first. After the tools run, _followups() re-plans. If the news call returned fewer than five articles, it adds a second fetch_news call with a widened "<company> stock" query.

In the graded run Gemini planned both runs. Run 1 had six steps: recall_memory, get_fundamentals, get_price_history (1-year period), fetch_news, get_edgar_filings, get_macro. Run 2 also had six steps but called get_edgar_filings before fetch_news, so in Run 2 EDGAR is [S4] and news is [S5]. The re-plan did not fire in either run because six of the twelve articles were relevant.

4.2 Dynamic tool use

Each tool wraps one source (see Table 1) and returns a ToolResult. fetch_news queries NewsAPI over a 14-day window with the ticker **and** a set of financial terms, then falls back to Yahoo Finance headlines if NewsAPI fails or returns nothing. SEC EDGAR needs no API key, only a contact User-

Agent under the SEC's fair-access policy. extra_tools sends SEC_USER_AGENT or, if it is unset, a generic default contact string. recall_memory is the retrieval tool for past runs. _call() retries a failed tool once, unless the error shows a missing key or an authorization failure. Section 7 adds retrieval over a FAISS index of the fetched articles.

In the graded run, news came from NewsAPI (12 articles) and FRED ran. SEC_USER_AGENT was not set, but the LLM planner chose get_edgar_filings, which ran with the default User-Agent and returned recent 10-Q and 8-K filings.

4.3 Self-reflection

Reflection works at two levels. The evaluator scores every draft against five criteria (Report Section 6.1: Grounding, Completeness, Balance, Transparency, Memory use) and returns specific feedback. After the loop ends, reflect() reviews the whole run: the final score, the number of refinements, the weakest criterion, and the number of data gaps. It writes an assessment and turns weak spots into lessons; an active LLM may rewrite the assessment and add up to three lessons. After Run 1, Gemini scored every criterion 5 out of 5 and wrote that "data limitations and baseline memory conditions were clearly delineated without introducing ungrounded speculation." Its three lessons came from what worked, not from a failure.

4.4 Learning across runs

After each run, MemoryStore saves the run's metrics, headlines, final feedback, scores, and note. Rule lessons carry tags: tool_failed:<tool>, news_thin (fewer than five usable articles), weak:<criterion> (a first-draft score below 4), and needed_refinement. Rule lessons are replaced on every run; LLM lessons accumulate up to the five newest. The next run uses memory in several ways: MemoryStore.diff() fills Changes since last run, citing [S1]; the lessons go to the LLM planner and writer; a news_thin lesson widens the news query in the rule plan; and in rule mode a weak:balance, weak:transparency, or weak:completeness lesson makes generate_draft() write the full note on the first try. The offline test test_learning_across_runs_offline checks the rule-mode pattern: Run 1 needs at least two evaluations and Run 2 passes on its first.

In the Gemini run, learning showed up differently: as memory recall, comparison with the prior run, and an adapted plan. Run 2's first step was "Retrieve prior run baselines, key metrics, and historical lessons learned for comparison," and its EDGAR step looked for "concrete operational headwinds," the wording of a Run 1 lesson. Its note named Run 1's ID under Changes since last run. The number of refinements varies with the LLM judge so it is not a reliable sign of learning in LLM mode.

5. Workflow Patterns

5.1 Prompt chaining

run_news_chain() runs five stages, each with a gate, and logs every stage: Ingest collects articles from every fetch_news result and records tool errors. Preprocess strips HTML, removes duplicate titles, and marks an article relevant only if it names the ticker (or the first word of the company name) **and** contains a financial keyword. We added the financial-keyword test after early runs pulled irrelevant articles that only mentioned the ticker. It keeps only relevant articles if at least three qualify; otherwise it keeps the deduplicated set so the chain never runs empty. Kept articles get IDs [N1], [N2], … Classify asks the LLM for a category (earnings, product, legal_regulatory, macro, analyst_rating, m_and_a, other) and a sentiment. A label is accepted only if both values are valid; otherwise keyword rules assign them.

Extract asks for up to three facts per article; with none, the title and regex-matched numbers are used. Summarize builds a rule digest (sentiment and category counts, notable titles). An LLM summary replaces it only if it cites at least one [N#] (single or combined, such as [N1, N3]) and every cited ID exists.

5.2 Routing
Router.route() sends price and macro results to the Market specialist. Fundamentals, filings, and articles classified as earnings or analyst_rating are sent to the Earnings & Fundamentals specialist. All other articles are sent to the News specialist. Each specialist returns a cited headline, findings, and risks. The Market specialist flags volatility above 35%, a price below both 20- and 50-day averages, or a price above 90% of its range. In the graded run AAPL was at 96%, so this flag fired. The Earnings specialist flags earnings within 14 days, a trailing P/E above 30, and negative earnings or analyst headlines. The News specialist reports the tone of non-earnings articles. If it doesn't have any, it checks the full chain output before saying no news was retrieved. In both graded runs, two articles went to Earnings (an analyst rating and an earnings story) and four to News.

5.3 Evaluator–optimizer
optimize() generates a draft, evaluates it, and, if it fails, refines it and evaluates again, for up to three refinements (four total evaluations). It stops early if a refinement returns unchanged text. refine() uses the LLM to address each feedback item from the evidence only or else inserts the missing sections from the specialists' output. The stored history (scores, feedback, overall, pass flag) is what Section 6b and Figure 2 display.

6. Evaluation and Iteration

6.1 Evaluator criteria and pass rule
The overall score is the mean of the five evaluation criteria. A draft passes when the mean is at least 4.0 and no criterion is below 3. When the Gemini judge is active, each of its scores can only lower the matching rule score (the lower of the two is kept), and up to five of its feedback items are added. The LLM therefore cannot inflate a score that the rule checks do not support, but it can lower one. In the graded run it agreed with the rule checks on every criterion. In an earlier run on October 17, it lowered Run 1's memory use to 4.

Criterion	Rule-based check in rule_evaluate() (score 1–5)
Grounding	Share of lines with numbers (outside Data gaps and Caveats) that carry a citation, single ([S2]) or combined ([S2, S3]), scaled to 1–5; capped at 2 if any cited ID is not a real source
Completeness	Share of the eight required sections present, scaled to 1–5. Headings are normalized (any level, bold, numbering, trailing colon, case). An unclosed "[" marks a truncated note and caps the score at 2
Balance	5 if Risks & Open Questions has at least one bullet ("-", "*", or "•"), otherwise 2
Transparency	If data gaps exist: 5 only if the Data gaps section names every gap, otherwise 2. With no gaps: 5 with the section, 4 without
Memory use	2 if a prior run exists but Changes since last run is empty or says "No prior run", otherwise 5

Table 3. Evaluation criteria.

6.2 Run 1 versus Run 2
Run 1 printed "Fresh start: cleared ira_state", so it is a true first run. Table 4 compares the runs.

Measure	Run 1	Run 2
Plan mode and steps	LLM, 6 steps	LLM, 6 steps (EDGAR before news)
Re-plan (widened news query)	No	No
News: ingested → kept (relevant)	12 → 6 (6 relevant)	12 → 6 (6 relevant)
Articles routed: Earnings / News	2 / 4	2 / 4
Evaluator iterations	1	1

Measure	Run 1	Run 2
First-draft score (weak criteria)	5.0 (none)	5.0 (none)
Final score, passed	5.0, passed	5.0, passed
Lessons saved after the run	3 LLM lessons: cite every figure; pair growth catalysts with concrete operational headwinds; document data limits and missing baselines. No rule lessons.	3 new LLM lessons; memory keeps the 5 newest unique ones (Run 2's 3 and 2 of Run 1's). No rule lessons.
Changes since last run	"No prior run to compare. [S1]"	"No change in tracked metrics since run 20261018T161147-1109c4 [S1]"
LLM calls, errors, truncated replies	7 calls, no errors, 0 truncated	7 more (14 in total), no errors, 0 truncated

Table 4. Run comparison from the evidence cell (Section 6b).

In rule mode the code produces a fixed pattern: Run 1's lean first draft fails, is refined, and saves weak:* lessons, and Run 2 passes on its first try (the offline test checks this). The Gemini run looks different. Both first drafts scored 5 on all five criteria, so the refine step was not needed and did not fire in this run. The loop did fire in our previous full run (October 17, 2026, about 6:40 PM PT): Run 2's first draft scored 4.4 and failed on memory use (2), because the writer was never told that a prior run existed, and one refinement raised it to 5.0. We traced that failure to compact() and fixed it (Table 5b). In the graded run, Run 2's first draft names Run 1's ID under Changes since last run and scores 5 on memory use without help from the refiner. So the fix to the writer's context did more than the refine loop. In LLM mode, learning shows up as memory recall, comparison with the prior run, and plan adaptation. The refinement count depends on the LLM judge. Both runs used the same market data (the October 16 close), so no tracked metric changed between them.

Two lines from the Run 2 Summary show the balance the Run 1 lessons asked for: "The stock trades near the high end of its range at 96% of its 251-session span [S3], supported by multiple growth catalysts including rapid sell-outs of the foldable iPhone Duo preorders [N1] and reported regulatory approval for Apple Intelligence in China [N2]." and "However, these tailwinds are directly balanced by concrete operational headwinds, specifically soft iPhone 18 demand and reported production cuts for the iPhone 18 Pro [N1][N3], alongside a premium valuation of 39.4x trailing earnings [S2]."

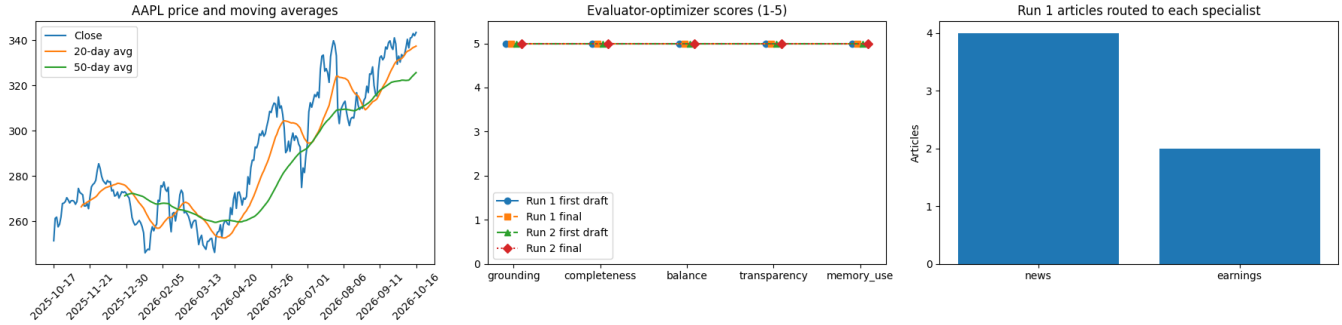


Figure 2. Visualization cell (Section 6c) output from the graded AAPL run. Left: closing price with 20- and 50-day averages over the one-year period the planner chose; the chart's last close (343.46) matches the close saved in memory. Middle: evaluator scores per criterion for the Run 1 and Run 2 first and final drafts. All four drafts scored 5 everywhere, so the lines overlap; small offsets keep each marker visible. Right: Run 1 articles routed to each specialist (4 News, 2 Earnings).

7. Integration Issues and Fixes

The two teammate notebooks were written separately and shared no modules. Merging them (Table 5a) and then running the agent with Gemini (Table 5b) exposed the issues below. Each fix is recorded in the module's docstring change log.

Issue	Root cause	Fix	Verification
Two notebooks, no shared code	Different data shapes (chunks vs. article dicts; earnings/general_noise label; Yahoo pubDate dropped)	One Colab wrapper writes 12 modules; article dicts in a ToolResult; general_noise mapped to other; FAISS kept as a side index; DeBERTa not loaded	Notebook runs top to bottom
"trading between of its averages"	pos = "between" inserted into "trading {pos} of its averages"	pos is a full phrase (above both / below both / between the two averages)	test_router_between_phrase
Invalid Claude model id	Default claude-sonnet-5-5 is not a real id	Changed to claude-sonnet-4-5; Claude runs only if ANTHROPIC_API_KEY is set	Code review (not used in runs)
News from NewsAPI only	Agent called fetch_news_newsapi directly; a missing key was a hard failure	Agent calls news_tools.fetch_news: NewsAPI first, Yahoo fallback	Graded run: NewsAPI, 12 articles
Synthetic "$85.8 billion" Apple article	News notebook inserted a hardcoded paragraph (fake URL, 0.98 confidence) when Yahoo returned nothing	Deleted; an empty fetch is ToolResult.failure, which becomes a data gap and a news_thin lesson	test_empty_yahoo_shape; test_tool_failure_is_disclosed
Irrelevant articles in the DIS run	Broad query on the bare ticker; preprocess fell back to keeping every article	Financial NewsAPI query; _is_financially_relevant post-filter; ticker/company AND keyword gate	Graded run: 12 fetched, 6 relevant
Summary "No news retrieved" beside listed articles	Earnings/analyst articles route to Earnings, leaving the News bucket empty	News specialist checks the full chain digest before saying no news was retrieved	No such line in either run
Offline test failed (3 != 5)	Stricter gate keeps 3 of 6 fixture articles	Assertion updated to 3	Ran 10 tests: OK

Table 5a. Issues found while merging the two notebooks.

Issue	Root cause	Fix	Verification
Gemini 404	The configured Gemini model had been retired, so every call failed and the agent silently used rules	Model read from GEMINI_MODEL, default gemini-3.8-flash	Graded run: every call answered
Gemini 429 quota	Free-tier request quota exhausted during a run (each run makes several LLM calls)	Existing backoff retries kept; moved to a paid API key	No errors in the graded run
Silent fallback to rules	Wrappers return "" on error; nothing showed which path ran	LLM status line prints model, calls, truncations, last error	Graded run: LLM mode, no errors
"Missing sections" on complete notes	sections() matched only exact "## Heading" lines; Gemini wrote "### Summary", "**Summary**", "## 1. Summary"	Headings normalized; writer prompt names the exact eight headings	Completeness 5 in both runs
Combined citations [S2, S3] ignored	Citation regex matched only [S2]; such lines counted as uncited, and LLM summaries	Evaluator (CITE_RE, cite_ids) and chain.summarize accept combined forms	Grounding 5; summary written by LLM

Issue	Root cause	Fix	Verification
	using [N1, N3] failed the chain gate		
"*" risk bullets not counted	Balance check counted only lines starting with "-"	Accepts "-", "*", and "•" bullets	Balance 5 in both runs
Truncated notes	4096 output tokens included Gemini's thinking tokens, so long notes stopped mid-sentence	Limit raised to 8192; MAX_TOKENS replies counted and reported; unclosed "[" caps completeness	0 truncated replies
Duplicate or unreadable lessons	A twice-failing tool saved the same lesson twice; the evidence print showed only the tag "llm"	Order-preserving de-duplication by (tag, text); print shows lesson text	Each lesson listed once with text
Writer not told a prior run existed	compact() sent only metric changes (empty when nothing moved) and no prior run id	compact() sends the Changes since last run lines with [S1] and the prior run id	Graded run: Run 2 names Run 1's id; memory use 5 on the first draft

Table 5b. Issues found in Gemini (LLM) mode.

7.1 Reflections on integration

Most of these issues did not cause the system to crash. The Gemini model, the empty Yahoo feed, and the empty News bucket each produced a plausible note through a fallback, so the only symptom was worse output. The fixes that mattered most were the ones that made the path visible: the LLM status line, the evidence cell, and the fail-closed data gaps. The Gemini-mode issues came from the gap between how the rule checks expected text to look and how the model actually wrote it. Each one was fixed by making the parser more tolerant or by telling the model exactly what format and context to use, not by changing the agent's design.

8. Lessons Learned

Fail closed. The synthetic article showed how a convenient fallback can put fabricated evidence into an index with high confidence. A visible data gap is better.
A shared contract makes integration possible. With one ToolResult shape and [S#]/[N#] IDs, the two notebooks could be joined without rewriting either author's logic.
Bugs hide in the fallback paths. The empty Yahoo feed, the retired model, and the empty news bucket all failed *quietly*, so each needs its own test or status line to review.
A rule checker must tolerate how an LLM writes. Strict heading, citation, and bullet parsing graded complete Gemini notes as missing sections.
Tell the LLM everything memory knows. In the October 17 run, Run 2 lost points because the writer saw no prior run, not because memory was empty. Once compact() passed the prior run ID, Run 2 passed on its first draft.
A perfect score is not proof. In the graded run the Gemini judge gave every draft 5/5, so the refine loop never fired. Showing that the loop works needed the earlier run and the offline tests.
Keep the code ticker-generic and run from a clean state. The first relevance fix had Disney-specific keywords; a reused memory file would have hidden the learning step.

9. Limitations and Future Work

Evaluator scores measure citation coverage, section completeness, and disclosure. The scores do not measure forecast accuracy and we specifically note this is not investment advice. The Gemini judge may be lenient or inconsistent, so the number of refinements varies between runs. In the graded run it gave every draft 5 out of 5, so the refine loop was not exercised. The graded run used the LLM path for

every step (14 calls, no errors, no truncated replies) and the deterministic rule path is verified by the offline tests as part of the fallback behavior. Both runs used the same market data, so no tracked metric changed. EDGAR ran with the default contact User-Agent because SEC_USER_AGENT was not set, and the SEC asks for a real contact. Yahoo headlines are often empty from Colab IP addresses, and NewsAPI looks back only 14 days, the code's default (days=14); NewsAPI itself allows about a month. Memory is lost when the runtime resets unless STATE points to Drive. The FAISS index is a retrieval demonstration and is not yet read by the chain. It indexed 20 chunks from the 12 NewsAPI articles and the Hugging Face token warning in Section 7 only means the embedding model was downloaded without authentication. Future work: Feed FAISS retrieval into the extract stage, and test the judge on deliberately weak drafts.

10. Team Contributions

Member	Contribution
Deepika Shrestha	Agent core: planning, tool layer, prompt chain, router and specialists, evaluator–optimizer, memory, and the original offline tests
Christian Lopez	News ingestion notebook: Yahoo Finance headline fetch (now the backup news source) and the FAISS search index (Section 7)
Cameron Aljilani	Integration of both notebooks into one Colab wrapper; bug fixes and Gemini-mode fixes; news-relevance filters; regression tests; evidence and visualization cells; executed Colab runs; final editing of the report
All	All members reviewed and contributed to the final notebook and this report.

11. AI Use Disclosure

Per the course AI policy, we disclose the following:
Cameron Aljilani: Used Grok (xAI) for integration, debugging, and documentation guidance: Colab setup; the news-relevance filters; diagnosing the “No news retrieved” bug and the failing offline test; the evidence and visualization cells; the Gemini-mode fixes in Table 5b (model configuration, quota handling, heading and citation parsing, bullet counting, truncation handling, lesson de-duplication, and the writer’s prior-run context), and making the workflow into a graphic.
Christian Lopez: Used Gemini (Google) for coding assistance on the news ingestion pipeline (News_Ingestion_Pipeline.ipynb), including the Yahoo Finance fetch, the classifier, and the FAISS index.
Deepika Shrestha: Used Claude (Anthropic) for coding assistance on the original agent notebook (Investment_Research_Agent_Deepika_Shrestha_09_29_2026_Github_Upload.ipynb), including the module structure, tools, workflows, and offline tests.
Google Gemini (gemini-3.8-flash, configurable with GEMINI_MODEL) via the google-genai SDK as an optional runtime component of the agent, used for planning, classification, extraction, summarization, drafting, judging, and reflection. Rule fallbacks run whenever it returns nothing.

AI output was treated as a draft: each code change was reviewed, edited, and tested (offline tests and the two executed runs) before it was kept, and every factual statement here was checked against the executed notebook. Our team understands and can explain the code, the design choices, and the results described here.

12. References

Aroussi, R. (n.d.). yfinance: Download market data from Yahoo! Finance's API [Computer software]. GitHub. https://github.com/ranaroussi/yfinance

Federal Reserve Bank of St. Louis. (n.d.). FRED API: fred/series/observations. https://fred.stlouisfed.org/docs/api/fred/series_observations.html

Google. (n.d.). Google Gen AI SDK for Python (google-genai) [Computer software]. https://googleapis.github.io/python-genai/

Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

NewsAPI. (n.d.). Everything endpoint documentation. https://newsapi.org/docs/endpoints/everything

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of EMNLP-IJCNLP 2019 (pp. 3982–3992). Association for Computational Linguistics.

U.S. Securities and Exchange Commission. (n.d.). EDGAR application programming interfaces. https://www.sec.gov/search-filings/edgar-application-programming-interfaces

University of San Diego. (2026). Presentation 7.1: Agentic AI [Course presentation]. AAI 520, Canvas. https://sandiego.instructure.com/courses/32494/pages/presentation-7-dot-1-agentic-ai

van Rossum, G., Warsaw, B., & Coghlan, N. (2001). PEP 8: Style guide for Python code. Python Software Foundation. https://peps.python.org/pep-0008/